SIMATS ENGINEERING
CO2_AT1_Scenario-Based Assessment
COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Question 1 - Online Banking: Fund-Transfer Boundary Failures

1. Problem Understanding and Context Analysis

The fund-transfer function accepts an amount and validates it against business rules before debiting the source account and crediting the beneficiary. Failures cluster at the **minimum and maximum limits**, which is the classic signature of an off-by-one or wrong relational operator (for example amount > 100 coded instead of amount >= 100). Because this is a financial transaction, a boundary defect means either a legitimate customer is blocked or an illegal over-limit transfer is allowed.

2. Evaluation of the Current Test Design Strategy

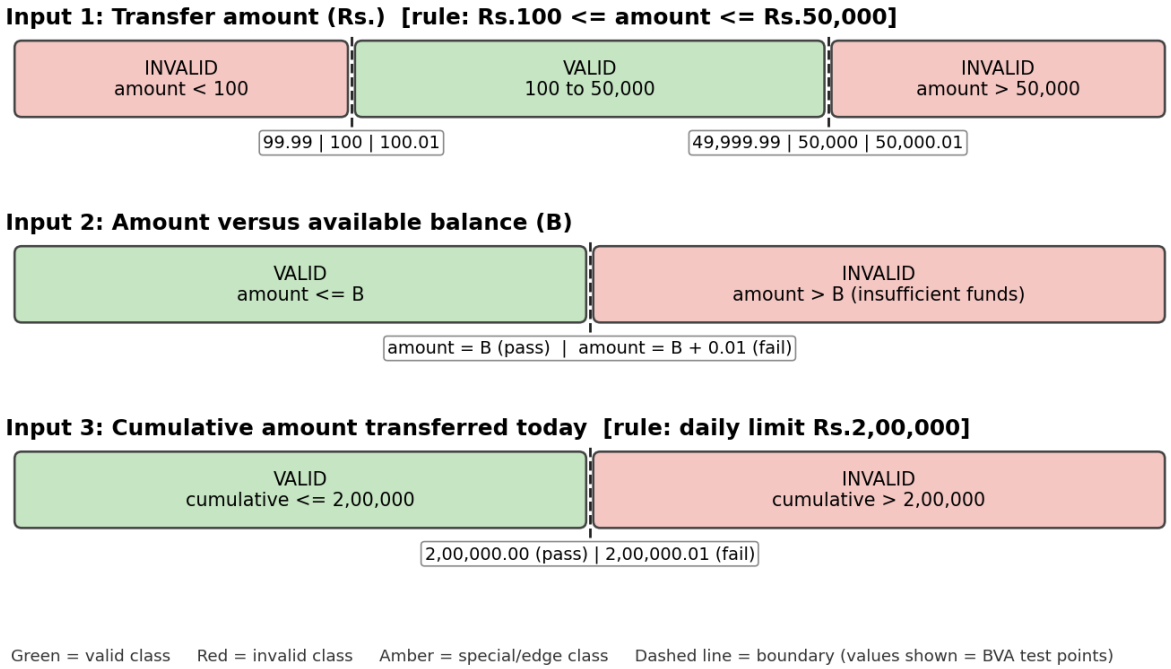
- **No systematic derivation:** tests were chosen by intuition or generated randomly, so there is no guarantee that every rule and every boundary is exercised.
- **Boundaries under-sampled:** random values almost never land exactly on 99.99, 100, 50,000 or 50,000.01, therefore the defective operators were never exposed before release.
- **Only one input considered:** amount was tested, but balance and daily-limit rules also have boundaries and they interact.
- **No negative testing and no adequacy criterion:** zero, negative, non-numeric and over-precision inputs were not covered, and nobody could say when testing was "enough".

3. Application of Black-Box Techniques

3.1 Equivalence Class Partitioning (ECP)

The input domain is divided into classes in which the system is expected to behave identically, so one representative per class is enough.

Fig 1.1 Equivalence classes and boundary values - fund transfer



Input	Valid classes	Invalid classes
Transfer amount	V1: 100.00 <= amount <= 50,000.00	I1: amount < 100; I2: amount > 50,000; I3: zero / negative; I4: non-numeric or empty; I5: more than 2 decimals
Balance rule	V2: amount <= available balance	I6: amount > available balance
Daily limit	V3: today's cumulative total <= 2,00,000	I7: cumulative total > 2,00,000

3.2 Boundary Value Analysis (BVA)

For every range the six boundary values **min-1, min, min+1, max-1, max, max+1** are tested (here with the smallest currency step of 0.01). This catches the wrong-operator and off-by-one defects that caused the production failures.

4.Proposed Structured and Optimized Testing Approach

Layer	Technique	Purpose	No. of tests
1	ECP on amount, balance, daily limit	One representative per valid and invalid class	10
2	BVA (6-point) on each limit	Expose off-by-one and operator defects	9
3	Decision table (R1-R4)	Verify rule combinations and rejection reasons	4
4	Error guessing / negative tests	Negative, empty, special characters, double-click, session timeout	6
5	Seeded random / fuzz regression	Breadth; repeat with fixed seed in CI	automated

Execution order: smoke (T4) -> boundary tests -> decision-table rules -> negative tests -> random regression. Every defect found is converted into a permanent regression test.

4.Justification and Conclusion

- **Maximum defect detection:** the failing region (the limits) is covered exactly, by 15+ targeted tests instead of thousands of random ones.
- **Optimization:** the number of whole-rupee amounts is 1,00,000, exhaustive testing is impossible; ECP + BVA reduces this to about 30 well-chosen tests with equal fault-finding power for boundary defects.
- **Measurable adequacy:** every class, every boundary and every decision-table rule is traceable, giving a clear exit criterion.

Conclusion: replacing random-first testing with ECP + BVA + decision-table design (with random testing as a supporting layer) removes the root cause of the boundary failures and gives a repeatable, auditable test suite for the banking module.

Question 2 - Healthcare Module: Insufficient Coverage

1. Problem Understanding and Context Analysis

The module is a **prescription / dosage validation** routine. It rejects invalid patient data, blocks drugs when an allergy is recorded, calculates a paediatric or adult dose, caps the dose at a safe maximum and approves the prescription. A defect in any of the rarely executed branches (allergy block, paediatric calculation, overdose cap) is patient-safety critical, which explains why the post-deployment defects were severe.

2. Evaluation of the Testing Approach and Adequacy Gaps

- Testing was **requirements-driven and happy-path oriented**: typical adult prescriptions and one invalid-input case were run.
- **No test adequacy criterion** (statement, branch, path or MC/DC) was defined or measured, so no one knew what code had never been executed.
- Safety-critical decisions (allergy, age < 12, dose > maximum) were never exercised; coverage tools were not used to find the gap.

3. Conceptual Control Flow Graph and Missing Paths

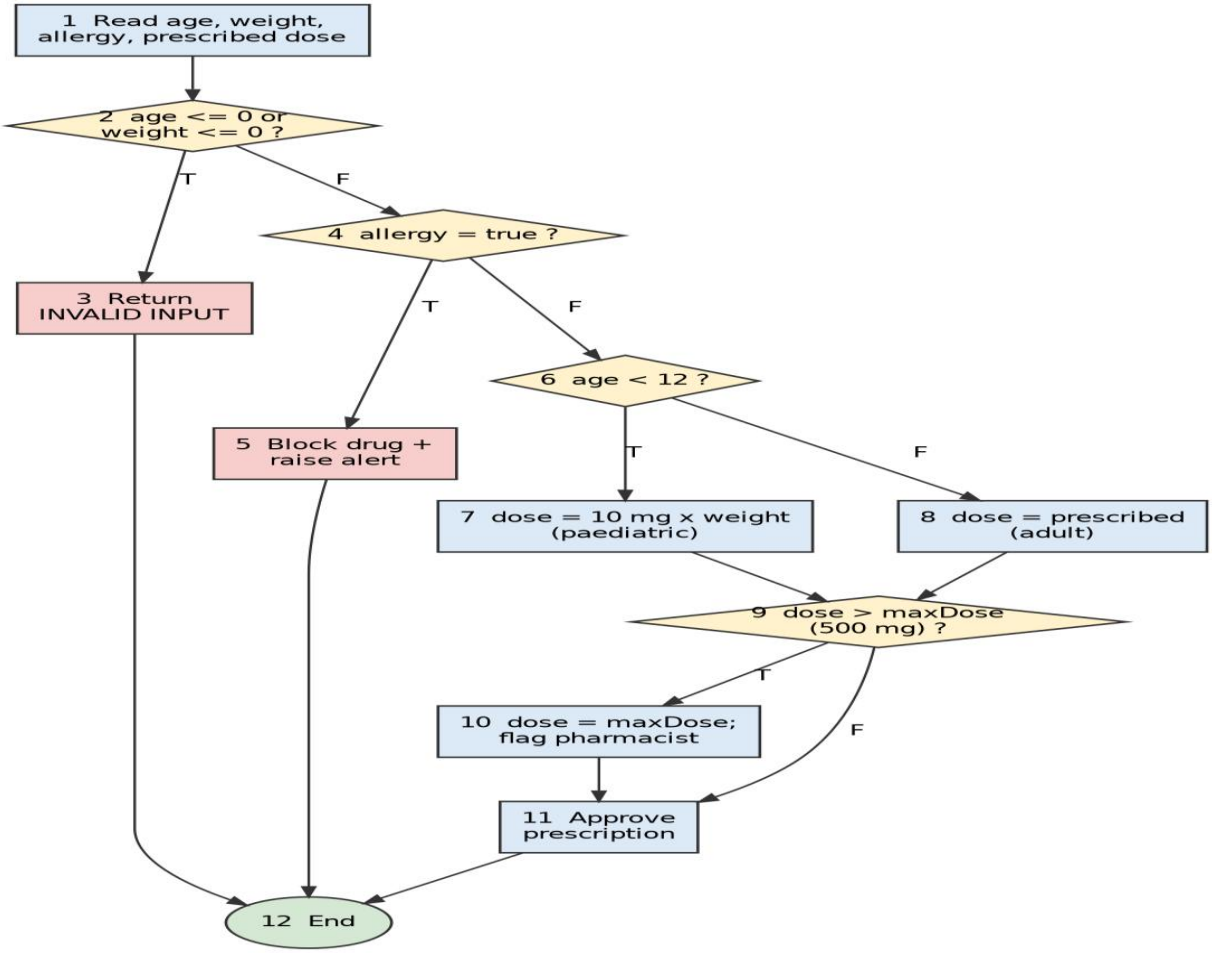


Fig 2.1 Control flow graph of the dosage-validation module (T = true, F = false)

Cyclomatic complexity: V(G) = E - N + 2 = 15 - 12 + 2 = **5** (also = predicates + 1 = 4 + 1). The graph has no loop, so the complete set of feasible paths is finite (6 paths).

Path	Node sequence	Meaning	Executed by old tests?
P1	1-2-3-12	Invalid input	Yes
P2	1-2-4-5-12	Allergy: drug blocked	NO
P3	1-2-4-6-7-9-11-12	Paediatric dose, within limit	NO
P4	1-2-4-6-7-9-10-11-12	Paediatric dose, capped	NO
P5	1-2-4-6-8-9-11-12	Adult dose, within limit	Yes
P6	1-2-4-6-8-9-10-11-12	Adult dose, capped	NO

4. Improved Testing Strategy (Complete Coverage)

4.1 Basis-path / full-path test set (assumed max dose 500 mg; paediatric dose = 10 mg x kg)

ID	Age	Weight (kg)	Allergy	Prescribed (mg)	Path	Expected result
T1	-1	70	N	400	P1	INVALID INPUT
T2	30	70	Y	400	P2	Drug blocked + alert
T3	8	25	N	-	P3	Dose 250 mg approved
T4	11	60	N	-	P4	Dose 600 -> capped 500 mg + pharmacist flag
T5	40	70	N	400	P5	Dose 400 mg approved
T6	40	70	N	600	P6	Capped 500 mg + pharmacist flag

4.2 Black-box boundary supplements

ID	Input	Expected
B1	age = 0	INVALID (age <= 0)
B2	age = 11 / age = 12 (weight 40, prescribed 400)	11: paediatric 400 mg; 12: adult 400 mg (switch at 12)
B3	adult prescribed 500 mg / 501 mg	500: approved, no flag; 501: capped to 500 + flag
B4	weight = 0 / 0.1	0: INVALID; 0.1: dose 1 mg approved

Question 3 - E-Commerce: Price, Quantity and Discount Combinations

1. Problem Understanding and Context Analysis

The order-total function computes **total = price x quantity x (1 - effective discount)**. Defects hide in the **interaction** of the three fields. **Assumed rules:** price Rs.1 - 10,000; quantity 1 - 100; discount 0 - 50%; quantity >= 10 earns an extra 5% bulk discount; the effective discount is capped at 50%, i.e. eff = min(d + bulk, 50).

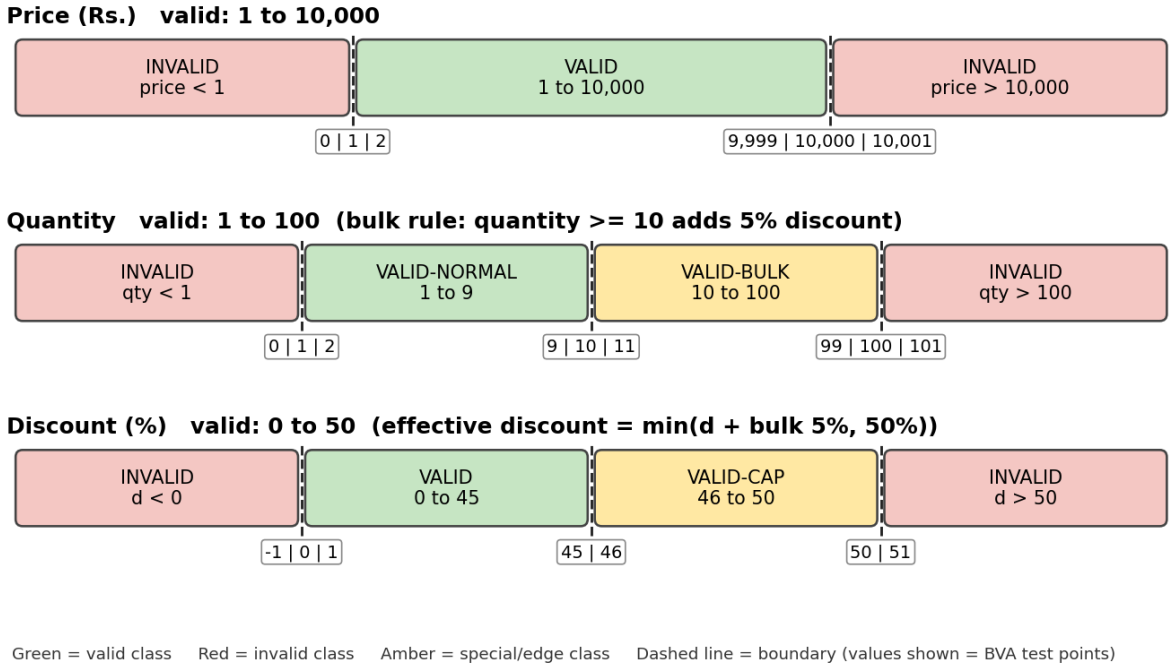
2. Limitations of the Current Black-Box Approach

- **Fields tested in isolation** (one at a time with the others fixed at "normal"), so interaction faults such as the bulk + discount cap were never triggered.
- **Weak coverage of partitions:** only one valid value per field was used; the bulk class (qty >= 10) and the cap class (d >= 46) were never combined.
- **Boundaries not combined:** BVA, if used, was applied per field but not on the dependent boundary where d + 5 crosses 50.
- **No decision logic model:** rules were not written as a decision table, so missing combinations were invisible.

3. Application of Black-Box Techniques

3.1 ECP and BVA

Fig 3.1 Equivalence classes and boundaries for price, quantity and discount



3.2 Strong (full-combination) ECP of the valid classes

Price has 1 valid class, quantity 2 (normal, bulk), discount 3 (0-45, 46-50 with 0). Valid combinations = 1 x 2 x 3 = **6**. Invalid classes are tested **one fault at a time** (so that one error does not mask another): price 3 + quantity 3 + discount 2 = **8**. Total ECP tests = 14.

3.3 Decision table for the effective discount

Rule	All inputs valid?	Quantity >= 10?	d + 5 > 50?	Effective discount / action
R1	No	-	-	Reject with field-specific error
R2	Yes	No	-	eff = d
R3	Yes	Yes	No	eff = d + 5
R4	Yes	Yes	Yes	eff = 50 (capped)

4. Justification

- **Completeness of combinations:** every class combination that has a different business rule (R1-R4) is exercised, including the dependent boundary at 45/46 with bulk quantity, which isolated testing can never hit.
- **Optimization:** an exhaustive boundary grid would need 7 x 9 x 9 = 567 tests; the design needs **21 tests** (6 + 7 + 8), a reduction of about 96% with the same fault coverage for the specified rules.
- **Better defect detection:** single-fault invalid tests avoid fault masking; boundary tests expose operator errors; the decision table exposes missing or wrongly ordered discount logic (e.g. cap applied before bulk).

Conclusion: combining strong ECP, BVA and decision-table testing replaces isolated field testing with a compact, complete and traceable suite that detects interaction defects among price, quantity and discount.

Question 4 - Nested Conditions and Loops: Partial Path Coverage

1. Problem Understanding and Context Analysis

The module processes a batch of n records. For each record of type 'A' it runs an inner loop over k items and either flags the record (amount > limit) or processes it normally; other record types are logged as skipped. A representative structure is used below; the same method applies to any real module.

2. Evaluation of the Current White-Box Approach

- Tests were written from the code's "main flow" rather than from a **control flow graph**, so decisions inside the inner loop and loop-exit conditions were missed.
- The CFG was incomplete: decision nodes, the loop back-edges and the loop-exit edges were not all drawn, which makes the count of independent paths wrong (cyclomatic complexity under-estimated).
- Only two independent paths were executed - a skipped-record path and a normal-processing path.

3. Control Flow Graph and Independent Paths

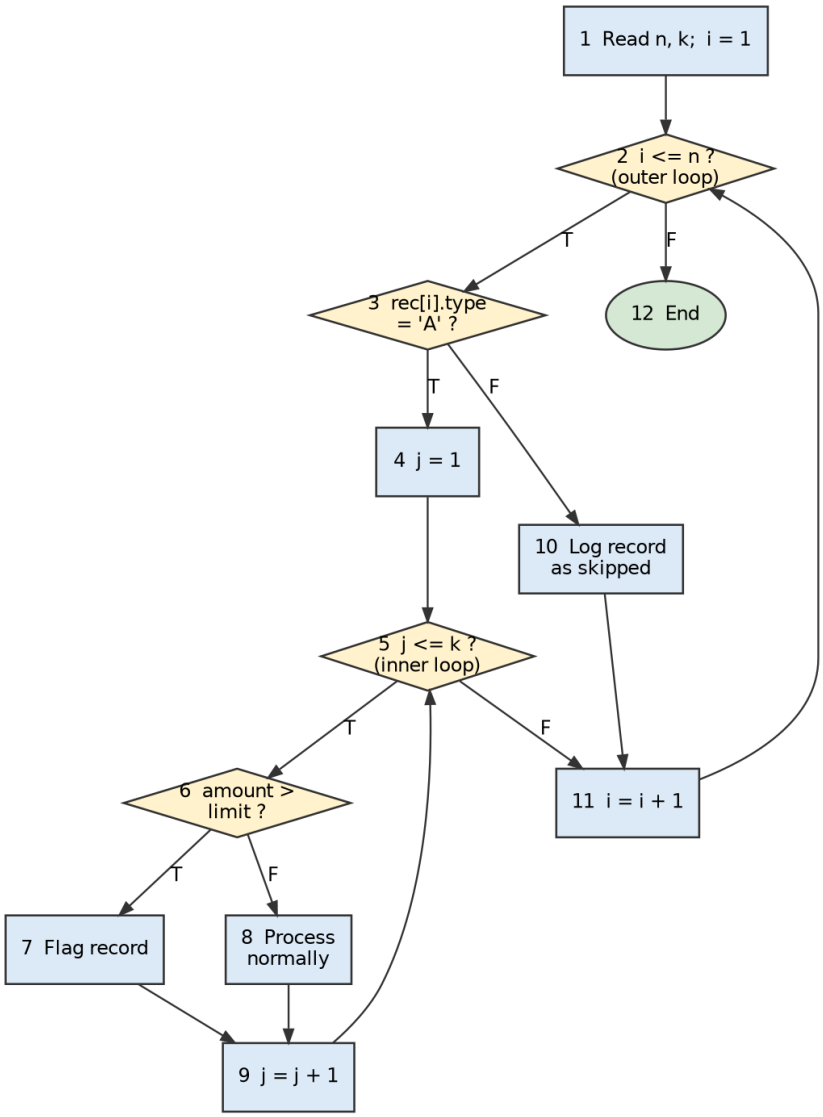


Fig 4.1 CFG of the nested loop module

Cyclomatic complexity: V(G) = E - N + 2 = 15 - 12 + 2 = **5**; check: decision nodes 2, 3, 5, 6 give P + 1 = 5. A set of **5 linearly independent (basis) paths** is therefore required.

4. Proposed Complete Path-Testing Strategy

4.1 Basis-path tests

ID	Path	Input	Expected outcome
TC1	P1	n = 0	No processing, normal termination, empty report
TC2	P2	n = 1, type = 'B'	Record logged as skipped
TC3	P3	n = 1, type = 'A', k = 0	No item processed; no error
TC4	P4	n = 1, 'A', k = 1, amount > limit	Record flagged
TC5	P5	n = 1, 'A', k = 1, amount <= limit	Processed normally

4.2 Loop-boundary and interaction tests (Beizer's loop testing)

ID	Target	Input	Expected outcome
TC6	Inner loop 2 iterations	n = 1, 'A', k = 2 (one over-limit, one normal)	One flagged, one processed
TC7	Inner loop max	n = 1, 'A', k = kmax; and kmax + 1	kmax handled; kmax + 1 rejected or handled safely
TC8	Outer loop max / max+1	n = nmax and nmax + 1	nmax handled; nmax + 1 rejected
TC9	Nested interaction / state reset	n = 3 records: 'A', 'B', 'A'; k = 2	Both 'A' records fully processed (j re-initialised)
TC10	Nested loops, outer at minimum	n = 1, k = 1, 2, kmax	Inner loop tested while outer is held constant

Because loops make the number of full paths unbounded, **complete path testing is replaced by basis-path coverage + loop boundary tests**, which is the standard feasible equivalent.

6. Justification Using Coverage Criteria and Expected Outcomes

- **Targets:** 100% statement, 100% branch, 100% of the 5 independent paths, plus 0 / 1 / 2 / max / max+1 iterations of each loop.
- **Expected outcome:** coverage moves from 91.7 / 87.5 / 40% to 100 / 100 / 100%; the latent defects (empty batch, k = 0, flag logic, state reset) become detectable.
- **Why V(G) is used:** V(G) = 5 is the minimum number of tests that guarantees every edge is executed; the 10 tests above add the loop and state cases that V(G) alone cannot reveal.
- **Maintainability:** if V(G) of the real module exceeds 10, refactor (extract the inner loop) to keep it testable.

Question 5 - Financial Application: Hybrid Black-Box + White-Box Strategy

1. Problem Understanding and Context Analysis

Both families of techniques are present but defects still escape. This indicates a **process and integration problem**, not a lack of techniques: black-box tests check requirements, white-box tests check code structure, and each finds faults the other cannot. Without an explicit link between them, gaps remain.

2. Proposed Hybrid Testing Strategy

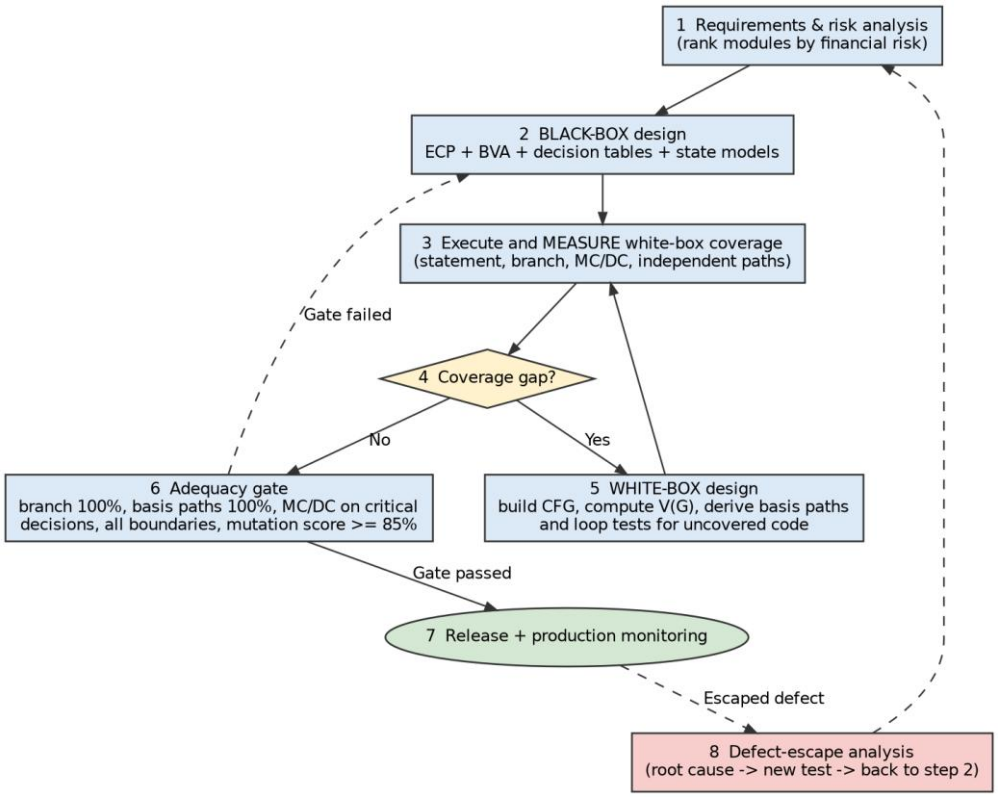


Fig 5.1 Hybrid black-box / white-box workflow with feedback loops

- **Step 1 - Risk ranking:** classify modules (interest, penalties, transfers = high risk; reports = low risk) and set criteria accordingly.
- **Step 2 - Black-box design:** ECP, BVA, decision tables and state models from the requirements.
- **Step 3 - Measure white-box coverage** of that suite with a coverage tool.
- **Step 4 - Gap analysis:** build the CFG, compute V(G) and list independent paths still unexecuted.
- **Step 5 - White-box design:** add basis-path and loop tests for the gaps, then trace every new test back to a requirement; a path that maps to no requirement signals dead or undocumented logic.
- **Step 6 - Adequacy gate and mutation testing** before release; **Step 7/8 - production feedback:** every escaped defect becomes a new test.

Worked illustration: loan-eligibility module

Rule: score must be 300-900 (else INVALID); APPROVE if score >= 700 **and** income >= 30,000, otherwise REJECT.

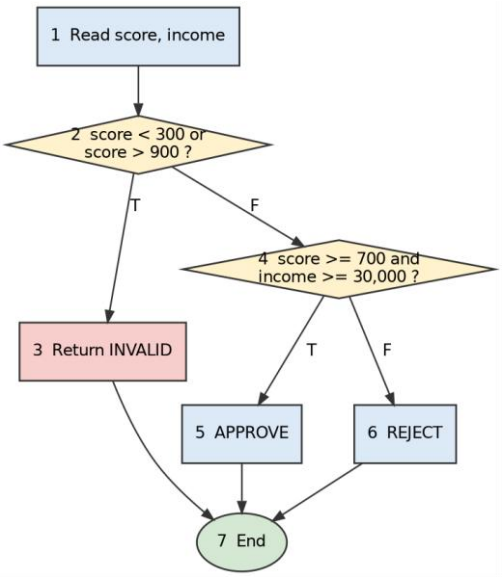


Fig 5.2 CFG of the loan-eligibility module, V(G) = 8 - 7 + 2 = 3

ID	Technique	Score	Income	Path	Expected
H1	BVA / MC/DC	299	40,000	1-2-3-7	INVALID
H2	BVA / MC/DC	901	40,000	1-2-3-7	INVALID
H3	ECP / MC/DC	750	40,000	1-2-4-5-7	APPROVE
H4	BVA / MC/DC	699	40,000	1-2-4-6-7	REJECT
H5	BVA / MC/DC	750	29,999	1-2-4-6-7	REJECT
H6	BVA exact limit	700	30,000	1-2-4-5-7	APPROVE (catches > vs >=)
H7	BVA	300 / 900	40,000	1-2-4-...	Valid range, not INVALID

H1, H2 and H3 together give MC/DC for decision 2 (3 tests for 2 conditions), H3, H4 and H5 give MC/DC for decision 4, and the three basis paths are all executed. Five tests thus satisfy **statement, branch, basis-path and MC/DC**, while H6-H7 add the black-box boundaries that coverage metrics alone cannot require.

3.Justification and Conclusion

- **Better coverage:** black-box tests give requirement coverage; white-box measurement shows what they missed; the added tests close it - each technique covers the other's blind spot.
- **Defect prevention:** the feedback loop converts each production defect into a test and a design rule, so the same class cannot recur; mutation testing proves the tests can actually fail.
- **Efficiency:** risk-based gates spend effort where financial loss is highest.

Conclusion: defects persisted because the techniques were used side by side but not integrated and the adequacy criteria were weak. The proposed risk-based, measured, feedback-driven hybrid (ECP/BVA + CFG/basis paths + MC/DC + mutation checks) provides traceable, verifiable coverage and reduces production escapes.

Question 6 - HR Payroll: Bonus Errors at 5, 10 and 15 Years

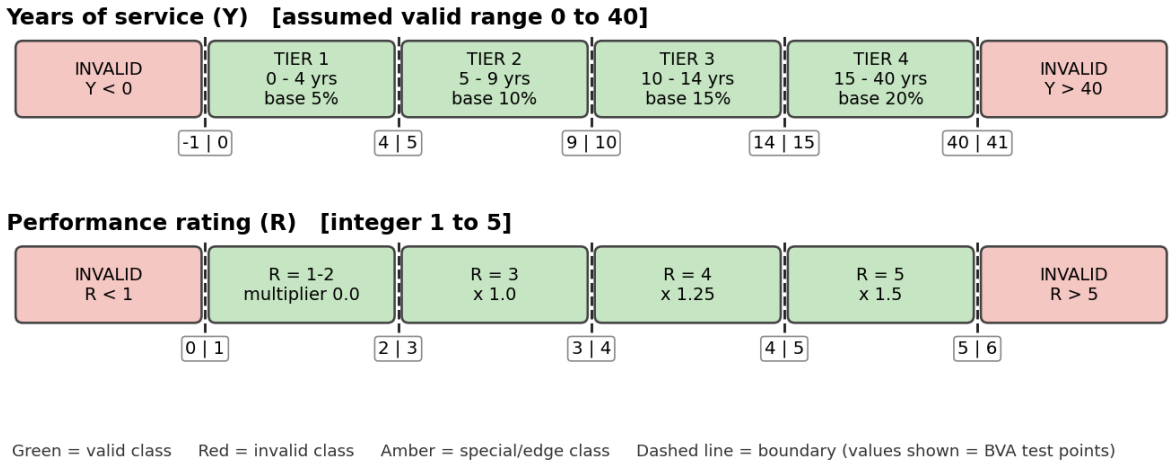
1. Problem Understanding and Context Analysis

Assumed rule: bonus = monthly salary x base % (by years of service) x rating multiplier. Base %: 0-4 years 5%, 5-9 years 10%, 10-14 years 15%, 15 years and above 20% (valid range 0-40). Rating (integer 1-5) multiplier: 1-2 = 0.0, 3 = 1.0, 4 = 1.25, 5 = 1.5.

Incorrect payouts at **exactly** 5, 10 and 15 years - while 4, 6, 9, 11 etc. are correct - is the textbook symptom of a **boundary defect**: the condition uses Y > 5 where Y >= 5 is required (or the reverse), so an employee on the threshold is placed in the lower tier and under-paid.

2. Application of ECP and BVA

Fig 6.1 ECP and BVA model for the payroll bonus rule



3. Redesigned Test Suite (monthly salary Rs.50,000)

3.1 Years of service (rating fixed at 3, multiplier 1.0)

ID	Years	Class / boundary	Expected %	Expected bonus (Rs.)
Y1	-1	invalid, below range	-	Error: invalid years
Y2	0	Tier 1 lower boundary	5%	2,500
Y3	1	Tier 1 min+1	5%	2,500
Y4	4	Tier 1 upper boundary	5%	2,500
Y5	5	**BOUNDARY Tier 2 (reported failure)**	10%	5,000
Y6	6	Tier 2 min+1	10%	5,000
Y7	9	Tier 2 upper boundary	10%	5,000
Y8	10	**BOUNDARY Tier 3 (reported failure)**	15%	7,500
Y9	11	Tier 3 min+1	15%	7,500
Y10	14	Tier 3 upper boundary	15%	7,500

ID	Years	Class / boundary	Expected %	Expected bonus (Rs.)
Y11	15	**BOUNDARY Tier 4 (reported failure)**	20%	10,000
Y12	16	Tier 4 min+1	20%	10,000
Y13	40	Tier 4 upper boundary	20%	10,000
Y14	41	invalid, above range	-	Error: invalid years

3.2 Rating (years fixed at 7, base 10%)

ID	Rating	Class / boundary	Expected bonus (Rs.)
R1	0	invalid	Error: invalid rating
R2	1	lower boundary of 0-bonus class	0
R3	2	upper boundary of 0-bonus class	0
R4	3	standard (boundary 2|3)	5,000
R5	4	boundary 3|4	6,250
R6	5	upper boundary	7,500
R7	6	invalid	Error: invalid rating

4.3 Combined boundary tests

ID	Years	Rating	Calculation	Expected bonus (Rs.)
C1	4	5	5% x 1.5 = 7.5%	3,750
C2	5	5	10% x 1.5 = 15%	7,500
C3	9	4	10% x 1.25 = 12.5%	6,250
C4	10	5	15% x 1.5 = 22.5%	11,250
C5	14	4	15% x 1.25 = 18.75%	9,375
C6	15	5	20% x 1.5 = 30%	15,000
C7	5	2	10% x 0 = 0%	0

Question 7 - Laboratory Temperature Control: MC/DC for Safety

1. Problem Understanding and Context Analysis

The controller reads the temperature, its rate of rise and the sensor status. Its logic (conceptual, thresholds assumed: LOW 15 C, HIGH 40 C, CRITICAL 60 C, rate limit 5 C/min) is:

- **Decision D1:** if sensorFault AND backupLost -> raise ALARM and safe shutdown.
- **Decision D2:** else if temp > HIGH -> cooler ON; **D3:** if temp > CRIT OR rate > 5 -> ALARM.
- **Decision D4:** else if temp < LOW -> heater ON; otherwise hold.

The reported field failure - no alarm in a particular sensor-malfunction sequence - is a defect in D1, whose true-branch is only exercised when **both** conditions are true.

2. Control Flow Graph

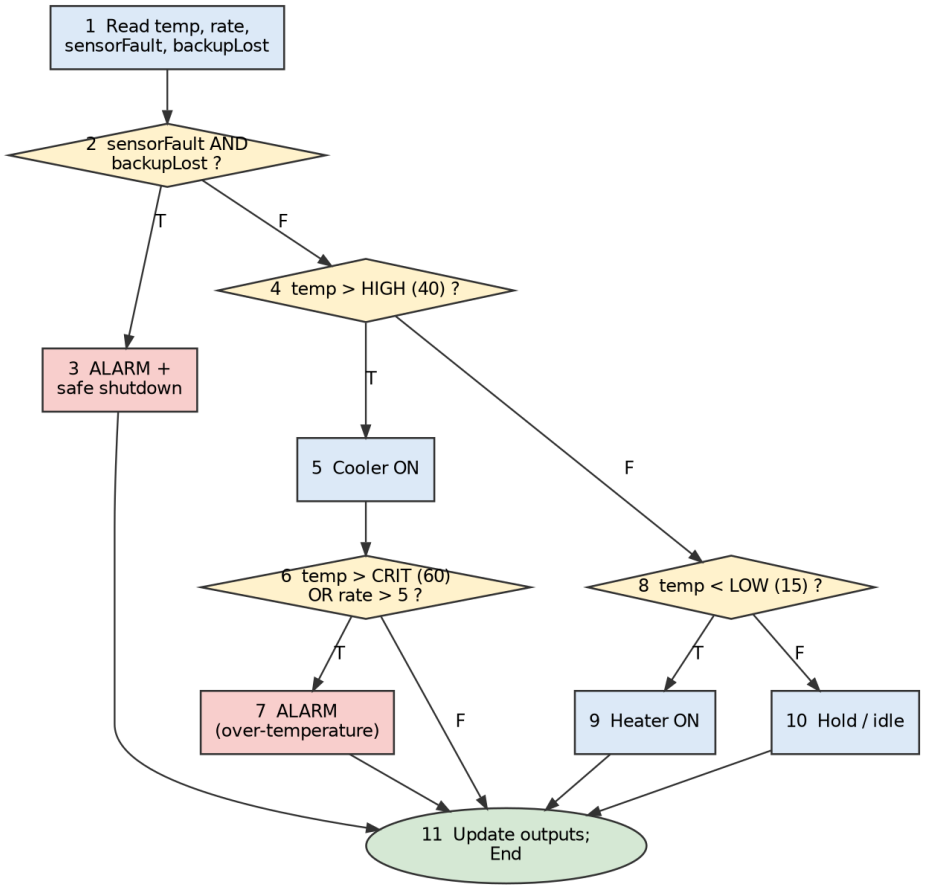


Fig 7.1 CFG of the temperature-control module

V(G) = E - N + 2 = 14 - 11 + 2 = **5** (decision nodes 2, 4, 6, 8). Counting atomic conditions, the module has 6 conditions in 4 decisions.

3. White-Box Strategy for 100% MC/DC

MC/DC requires: every decision takes both outcomes, every condition takes both values, and each condition independently affects the decision outcome (a pair of tests differing only in that condition and giving different results). A decision with n conditions needs n + 1 tests.

3.1 Combined test set (7 tests give 100% MC/DC; T8 adds the sequence)

ID	SF	BL	Temp	Rate	Expected output	MC/DC role
T1	T	T	25	0	ALARM + shutdown	D1 = TT (True)
T2	F	T	25	0	Idle, no alarm	D1 = FT (SF flips); D2 F, D4 F
T3	T	F	25	0	Idle, no alarm	D1 = TF (BL flips)
T4	F	F	45	1	Cooler ON, no alarm	D2 T; D3 = FF
T5	F	F	65	1	Cooler ON + ALARM	D3 = TF (C flips vs T4)
T6	F	F	45	8	Cooler ON + ALARM	D3 = FT (D flips vs T4)
T7	F	F	10	0	Heater ON	D4 True
T8	seq	seq	25	0	Cycle 1 (SF=T, BL=F): backup in use; cycle 2 (BL=T): ALARM	Sequence test for the reported field failure

Supporting measures: boundary tests on thresholds (temp = 40, 41, 15, 14, 60, 61; rate = 5, 5.1), fault-injection of sensor failures, and a structural coverage report proving that every condition has an independence pair.

4. Justification: Impact on Safety-Critical Reliability

- **Detects masked faults:** MC/DC kills AND/OR operator mutants and missing-condition defects that branch coverage cannot, which is why it is mandated for the highest software levels in aviation (DO-178C Level A) and recommended in other safety standards.
- **Efficient:** n + 1 tests per decision (10 required, 7 after sharing) instead of 2^n; the exhaustive truth table of all 6 conditions would require 64 tests.
- **Direct fix of the incident:** T1-T3 and T8 pin down the alarm decision; any regression of D1 is caught automatically.
- **Safety argument:** demonstrable 100% MC/DC on alarm and shutdown logic gives auditable evidence that the alarm cannot be silently disabled by a single-condition fault.

Question 8 - Travel Portal: Promo-Code Combinations

1. Problem Understanding and Context Analysis

Assumed rules: a flat promo gives Rs.1,000 off, a percentage promo gives 20% off (percentage is applied first, then flat); Gold-tier users get a further 5% after promotions; the total payable must never fall below the minimum payable (Rs.49 convenience fee). The negative total arises when the combined discount exceeds the ticket price and the code has no floor.

2. Limitations of Testing Inputs in Isolation

- Each code was tested alone on a high price, where discount < price, so the negative region was never entered.
- The failure depends on the **interaction** of three variables (price, promo combination, tier); isolated tests can only reveal single-input faults.

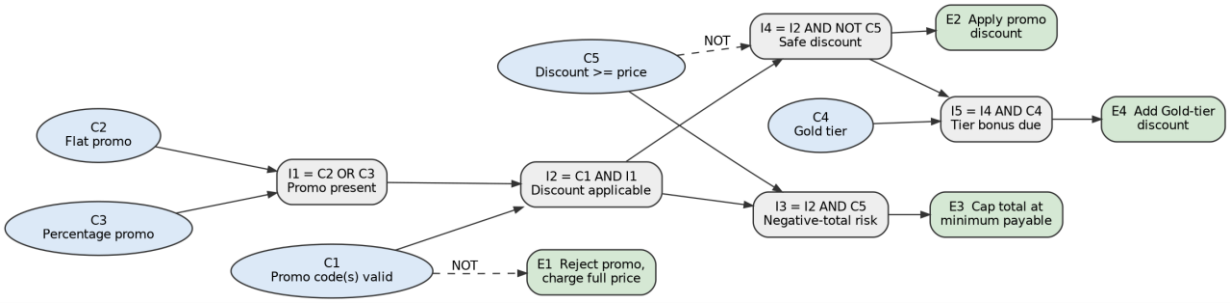


Fig 8.1 Cause-effect graph (dashed edge = logical NOT)

4. Optimized Test Set

Test	Rule	Price (Rs.)	Promo	Tier	Calculation	Expected total
TC1	R1	5,000	invalid code	Standard	promo rejected	5,000
TC2	R2	5,000	Flat 1,000	Standard	5,000 - 1,000	4,000
TC3	R3	800	Flat 1,000	Standard	800 - 1,000 < 0 -> floor	49
TC4	R4	5,000	20%	Standard	5,000 x 0.8	4,000
TC5	R5	5,000	20% + Flat	Standard	4,000 - 1,000	3,000
TC6	R6	1,000	20% + Flat	Standard	800 - 1,000 = -200 -> floor	49

Supplementary: BVA on the floor (discount = price - 1, = price, = price + 1), expired code, the same code applied twice, and a mis-configured 100% promo.

5. Justification

- **Maximum defect detection:** all four negative-total rules (R3, R6, R8, R11) are tested, including the exact boundary where discount = price (TC8).
- **Minimum redundancy:** 5 binary-style conditions give 32 raw combinations; collapsing invalid codes into R1 and removing infeasible cells leaves **11 tests** (about 66% fewer) with every distinct business rule covered once.
- **Traceability:** each test maps to one rule, so a failing test points directly to the faulty rule.

Question 9 - Database Migration Tool: Infinite Loops Despite Statement Coverage

1. Problem Understanding and Context Analysis

The tool iterates over N records and, for each valid record, over F fields. When a field is null it calls repair(field) but the loop counter j is not incremented on that path (the increment exists only in the non-null branch). If the repair does not make the field non-null, the loop returns to the same field forever. A representative CFG is shown below.

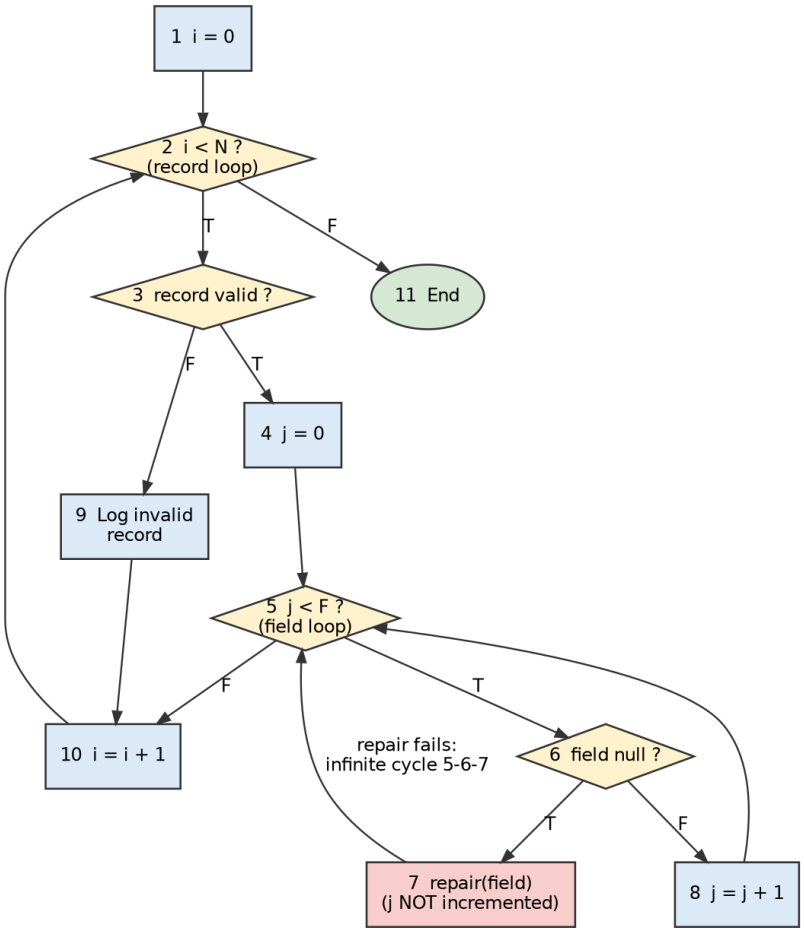


Fig 9.1 CFG of the record-validation routine (defective back-edge 7 -> 5 marked)

2. Cyclomatic Complexity and Independent Paths

V(G) = E - N + 2 = 14 - 11 + 2 = **5**; verification: decision nodes 2, 3, 5, 6 -> 4 + 1 = 5. Five linearly independent paths must be executed:

Path	Node sequence	Scenario
P1	1-2-11	N = 0
P2	1-2-3-9-10-2-11	Invalid record
P3	1-2-3-4-5-10-2-11	Valid record, F = 0 (inner loop skipped)
P4	1-2-3-4-5-6-8-5-10-2-11	Valid, non-null field
P5	1-2-3-4-5-6-7-5-10-2-11	Valid, null field, repair succeeds

3. Proposed Path-Testing Strategy

ID	Target	Input	Expected outcome
TC1	P1	N = 0	Clean exit
TC2	P2	N = 1, invalid record	Record logged, loop ends
TC3	P3	N = 1, valid, F = 0	No field checks, continue
TC4	P4	N = 1, F = 1, not null	Field accepted
TC5	P5	N = 1, F = 1, null, repair ok	Repaired, advance to next field
TC6	Loop stress (the defect)	F = 1, null, repair stub always fails; timeout / max-iteration guard	Terminates with "repair failed" error after retry limit; **fails on the old code**
TC7	Inner cycle twice	repair succeeds on 2nd attempt	Path 5-6-7-5-6-8: terminates
TC8	Inner loop boundaries	F = 0, 1, 2, Fmax, Fmax + 1	Correct counts; Fmax + 1 rejected
TC9	Outer loop boundaries	N = 0, 1, 2, Nmax, Nmax + 1	Correct counts; Nmax + 1 handled
TC10	Nested interaction	N = 3; F = 3 with null in field 2 of record 2	Only the null field repaired; j and i reset correctly

5. Justification Using Cyclomatic Complexity

- The sample routine has V(G) = 5, therefore **at least 5 path tests** are mandatory; the whole legacy tool with deeper nesting will have a larger V(G) which sets its minimum test count and flags it for refactoring (extract the field loop into its own function).
- Coverage moves from 100% statement / 0% on the defective cycle to **100% statement, 100% branch, 100% independent paths and all loop boundaries** - the criteria that directly address "infinite loop" defects.

Question 10 - Food Delivery Tracker: State Transition Testing

1. Problem Understanding and Context Analysis

The tracker is a **finite-state machine**: its behaviour depends on the current state and the event received, not only on input values. The specification covers only the happy path (Ordered -> Preparing -> Out for Delivery -> Delivered). The driver-cancel event during delivery has no defined target state, so the UI stays on "Out for Delivery" indefinitely.

2. State Transition Model

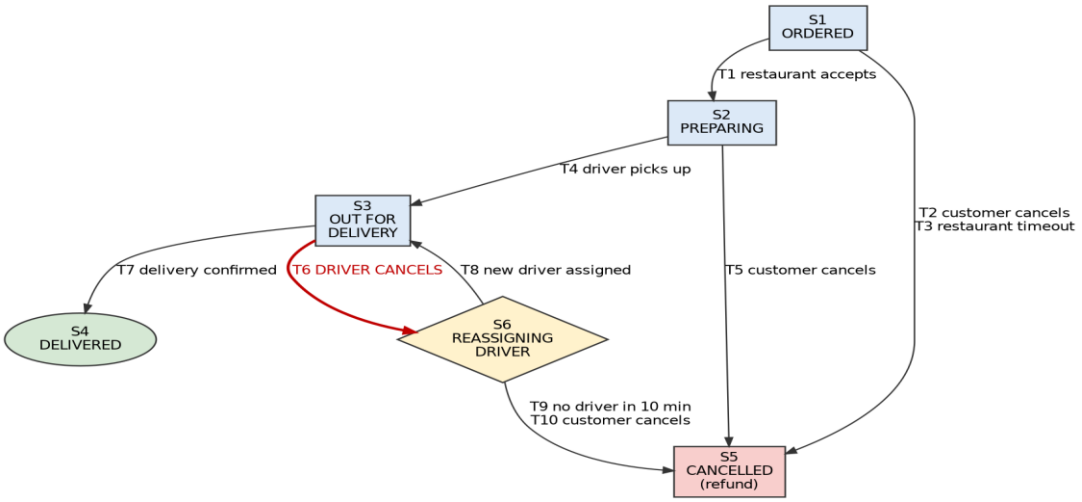


Fig 10.1 State transition diagram including the driver-cancel flow (red = defect scenario)

3. Justification: Gaps Missed by ECP and BVA

Aspect	ECP / BVA	State transition testing
What it models	Value ranges of one input at a time	Behaviour that depends on history: state + event
Sequence of events	Not represented	Explicit transitions and paths (0-switch, 1-switch)
Missing states / transitions	Cannot detect an unspecified event	Table shows every undefined state-event cell
Invalid transitions	Not considered	Systematically tested (32 invalid cells)
Reported defect	No input value is wrong - cannot reveal it	T6 / T8 are explicit tests

Conclusion: ECP and BVA remain useful for order fields (address, amount), but the stuck-tracker defect is a **workflow defect**. Modelling the tracker as a state machine exposes the missing Reassigning state, provides complete transition coverage, and prevents the same class of defect in other dynamic flows.

Question 11 - Authentication Module: Hybrid Testing against Brute Force

1. Problem Understanding and Context Analysis

Login succeeds only if the account is not locked, the password is valid, the OTP is correct **and** within 60 seconds, and the IP is trusted (otherwise step-up verificationBrute-force weaknesses arise when a guessing channel is not rate-limited, when counters can be bypassed, or when OTP/password checks can be repeated without penalty.

2. Formal Path Analysis

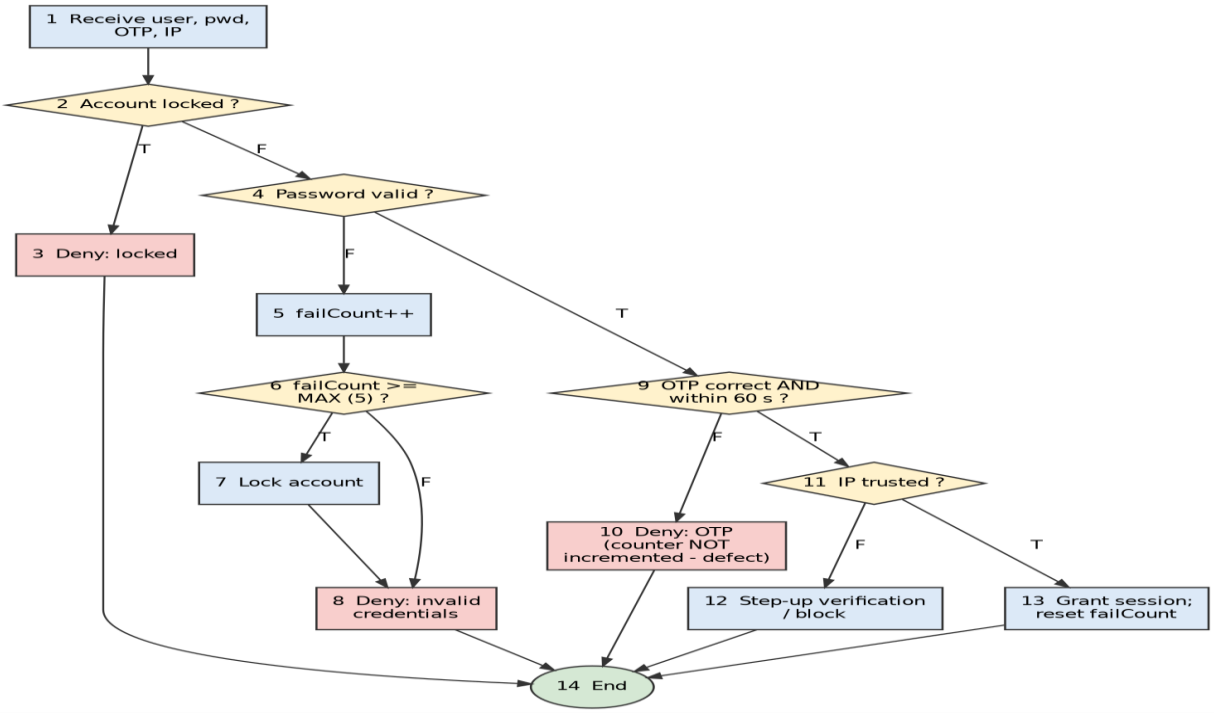


Fig 11.1 CFG of the authentication routine

V(G) = E - N + 2 = 18 - 14 + 2 = **6** (decisions 2, 4, 6, 9, 11). Six independent paths:

Path	Node sequence	Meaning	Security observation
P1	1-2-3-14	Account locked	Correct
P2	1-2-4-5-6-8-14	Wrong password, below limit	Counter incremented
P3	1-2-4-5-6-7-8-14	Wrong password, limit reached	Account locked
P4	1-2-4-9-10-14	Correct password, wrong / expired OTP	**No counter increment: unlimited OTP guesses (brute force)**
P5	1-2-4-9-11-12-14	Untrusted IP	Step-up required

4. Justification: Improved Security Posture

- **Error guessing** supplies attacker thinking that no specification contains (OTP replay, races, IP spoofing, enumeration), covering **abuse cases**, not only use cases.
- **Combined effect:** paths show where controls are absent; guessing shows how an attacker would exploit that absence. Fixes (shared counter on all failure paths, lockout/back-off, OTP single-use, atomic counter) are verified by H3-H5 and E1-E4 and kept as regression tests.

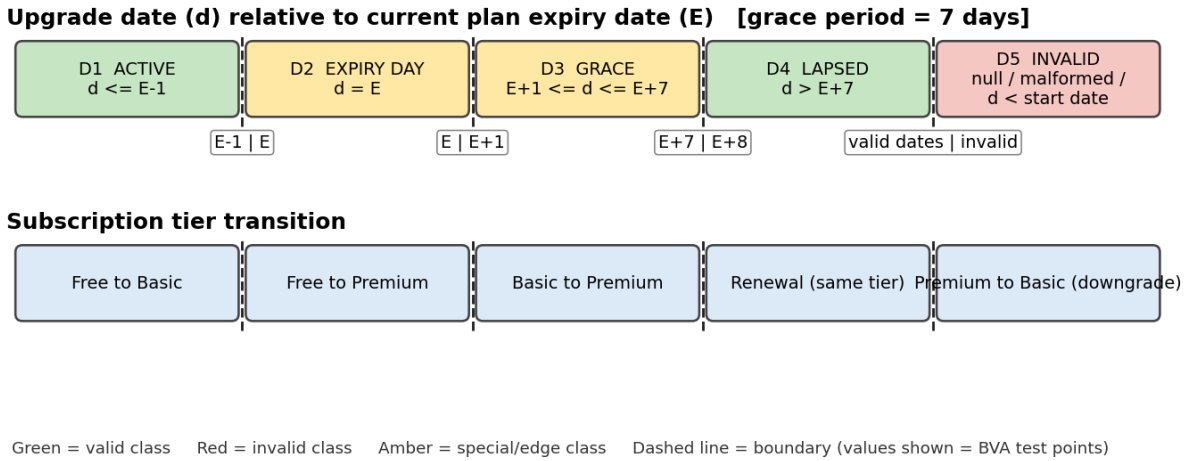
Question 12 - E-Learning Platform: Upgrade on the Day of Expiration

1. Problem Understanding and Context Analysis

Access rights depend on the subscription tier, its expiry date E and the course type (Free, Basic or Premium course). The error at "upgrade exactly on expiry day" indicates an inconsistent treatment of the boundary date: one component treats day E as expired (date >= E), another as active (date <= E),

2. Equivalence Class Partitioning

Fig 12.1 Equivalence classes for dates and subscription tiers



Class	Definition (d = upgrade date, E = expiry)	Representative / boundary values	Expected behaviour (assumed rule)
D1 Active	start <= d <= E - 1	E - 30, E - 1	Immediate upgrade; unused days credited
D2 Expiry day	d = E (any time up to 23:59:59)	E 00:00:00, E 12:00, E 23:59:59	Plan still active: upgrade without gap or double charge
D3 Grace	E + 1 <= d <= E + 7	E + 1, E + 7	Treated as renewal + upgrade at full price; access continuous
D4 Lapsed	d > E + 7	E + 8, E + 90	New subscription; access from payment time
D5 Invalid	null, malformed, d < start date	null, 31-Feb, start - 1	Rejected with message; tier unchanged

5. Justification of Completeness

- **Date axis complete:** the domain of dates is partitioned into disjoint classes that cover it entirely, with boundaries E - 1, E, E + 1, E + 7, E + 8 tested - exactly where defects such as < versus <= occur.
- **Tier axis complete:** all six legal transitions are represented and the illegal / invalid-date case is rejected.

Conclusion: the structured ECP design with boundary dates guarantees seamless transitions: users keep access, are billed once and receive the correct expiry date in every date class.

Question 13 - Smart-Home Lighting Controller: Branch and Condition Coverage

1. Problem Understanding and Context Analysis

The controller selects a dimming level by mode (AUTO, MANUAL, NIGHT, default). In AUTO mode the level is 100 - lux when lux < 100 AND occupied, otherwise 0. A final safety rule caps the level at 80% when level > 80 AND NOT override to protect the LED driver. The resulting PWM value is sent to hardware.

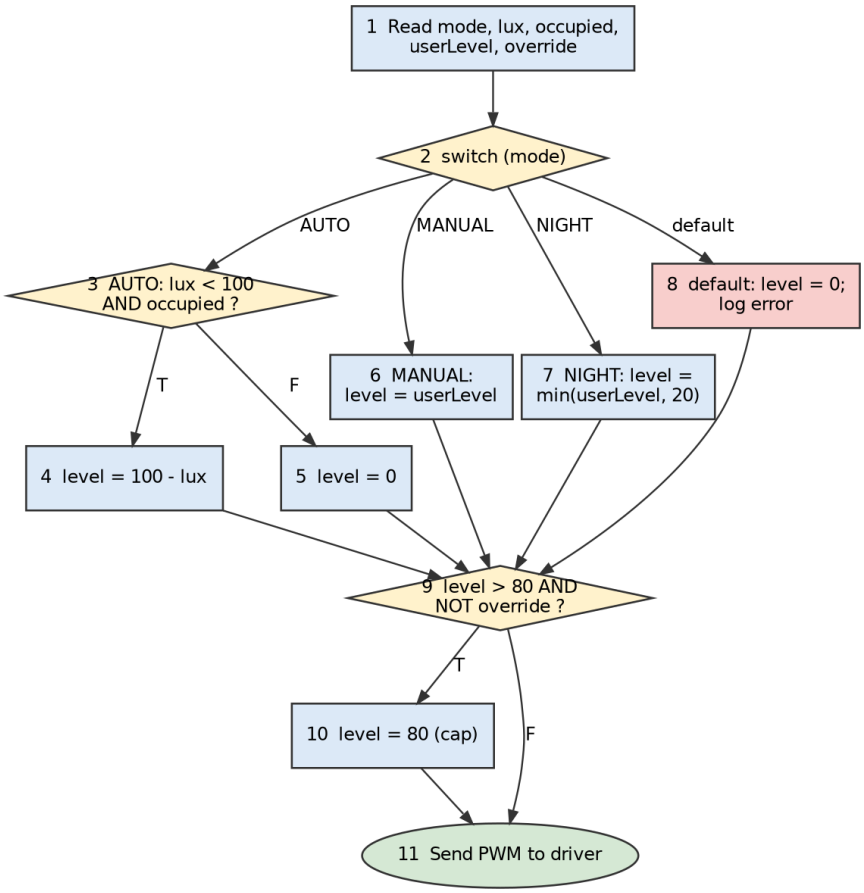


Fig 13.1 CFG of the dimming algorithm (V(G) = 15 - 11 + 2 = 6)

2. Risks of Relying Solely on Statement Coverage

- A statement is "covered" when it executes once; the **truth value of each Boolean condition** is irrelevant, so compound expressions can be only half tested.
- A switch with a missing or wrongly ordered case/break can pass statement coverage if every line runs once in some test.
- Hardware-related faults (wrong comparison operators on lux thresholds, cap not applied when override is set) are invisible to the metric.

Demonstration - a statement-oriented suite of 5 tests:

3.Improved White-Box Strategy (Branch + Condition + MC/DC + boundaries)

MC/DC pairs: decision 3 (A = lux < 100, B = occupied): W1 (TT), W2 (FT), W3 (TF). Decision 9 (C = level > 80, D = NOT override): W4 (TT), W6 (FT), W5 (TF).

ID	Mode	lux	occ.	userLevel	override	Expected level	Purpose
W1	AUTO	50	T	-	F	50	A=T, B=T

ID	Mode	lux	occ.	userLevel	override	Expected level	Purpose
W2	AUTO	150	T	-	F	0	A flips
W3	AUTO	50	F	-	F	0	B flips
W4	MANUAL	-	-	90	F	80	C=T, D=T: cap
W5	MANUAL	-	-	90	T	90	D flips: override
W6	MANUAL	-	-	70	F	70	C flips
W7	NIGHT	-	-	60	F	20	NIGHT limit
W8	invalid	-	-	-	F	0 + log	default case
W9	AUTO	99	T	-	F	1	BVA lux
W10	AUTO	100	T	-	F	0	BVA lux
W11	MANUAL	-	-	80	F	80	BVA cap
W12	MANUAL	-	-	81	F	80	BVA cap
W13	AUTO	0	T	-	F	80	path 4-9-10
W14	AUTO	-5 / 65535	T	-	F	safe state (0), fault logged	sensor out of range

5. Justification: Reduced Hardware-Software Integration Risk

- **Conditions verified individually:** MC/DC proves that each sensor/state condition really controls the dimming decision, so a wrong operator or missing condition is detected.
- **Hardware limits protected:** W4, W5, W12, W13 test the 80% cap and its override, the exact behaviour that protects the driver from over-current.
- **Sensor faults:** W14 and the boundary tests verify safe behaviour when lux readings are out of range or noisy.
- **Cost is modest:** 14 tests instead of the 2^4 x 4 = 64 input combinations of the decision logic, and the suite is automatable on a hardware-in-the-loop rig.

Question 14 - Logistics: Zero and Negative Weight Inputs

1. Problem Understanding and Context Analysis

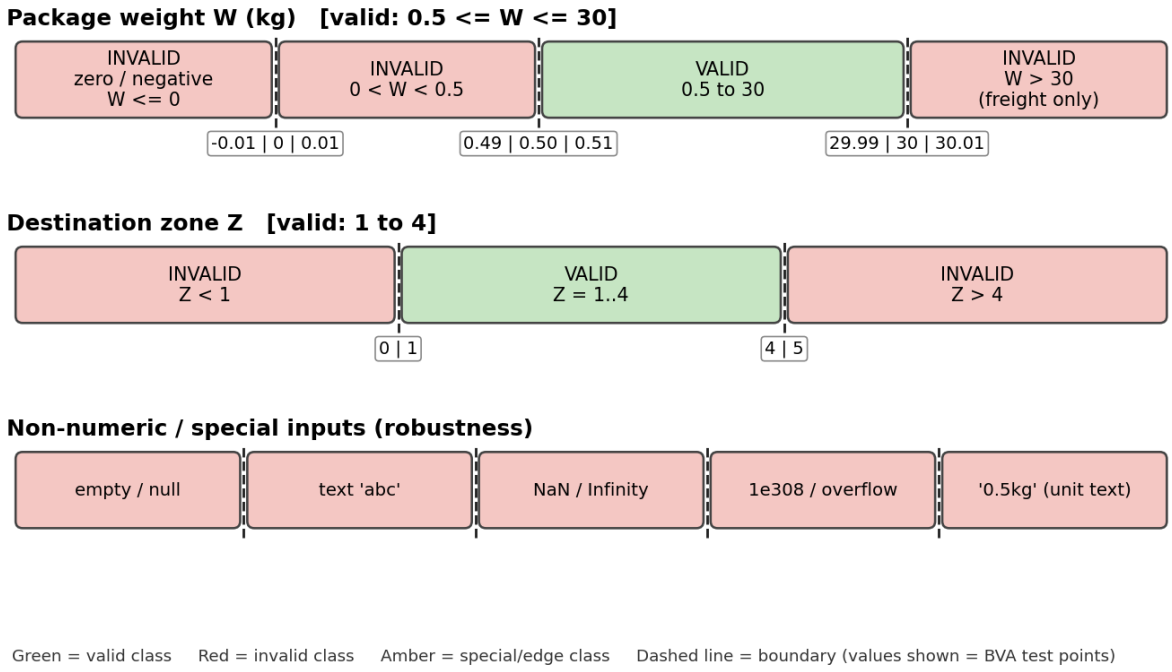
Assumed rule: shipping cost = base(zone) + rate x weight x speed factor; valid weight 0.5 kg to 30 kg, zone 1-4, speed in {Standard, Express, Overnight}. With zero weight the weight-dependent part is 0, so a parcel ships for the base fee (or free); with negative weight the formula produces a **reduced or negative cost**, which could credit the customer or crash a downstream billing step.

2. Analysis of the Failure of Current Input-Validation Testing

- Tests were **positive-only** (normal-boundary BVA, "does it work?"); nobody asked "what if the input is wrong?".
- Standard BVA uses only valid boundaries (min, min+1, max-1, max); robustness (worst-case) boundaries just outside the range were not selected.

3. Robustness Boundary Value Analysis

Fig 14.1 Robustness boundary value analysis (valid and invalid boundaries)



5. Justification: Why Negative Scenarios Are Necessary

Reason	Explanation
Financial integrity	Zero or negative weights change revenue: free shipping, negative charges, refund exploitation.
Robustness / stability	Unhandled invalid input causes exceptions, division errors, crashes in downstream systems.
Security	Invalid input is the main attack vector (injection, overflow); validation must be verified, not assumed.
Data quality	Bad weights corrupt carrier manifests, load planning and analytics.
Real-world behaviour	Users, scanners and API clients do send wrong data; defects at invalid boundaries are the most common production failures.

Question 15 - Medical Imaging: Prime Data Sizes in Parallel Loops

1. Problem Understanding and Context Analysis

The data (N elements) is divided among T threads. The code computes chunk = N / T and rem = N mod T. If rem = 0 the work is split evenly; otherwise a remainder-handling branch distributes the leftover elements. A **prime** N is divisible only by 1 and itself, so for every thread count T from 2 to N - 1, rem is never 0 - the remainder branch (with its off-by-one or dropped-slice defect) is **always** executed. For typical sizes (powers of two, multiples of 8 or 16) this branch is almost never reached.

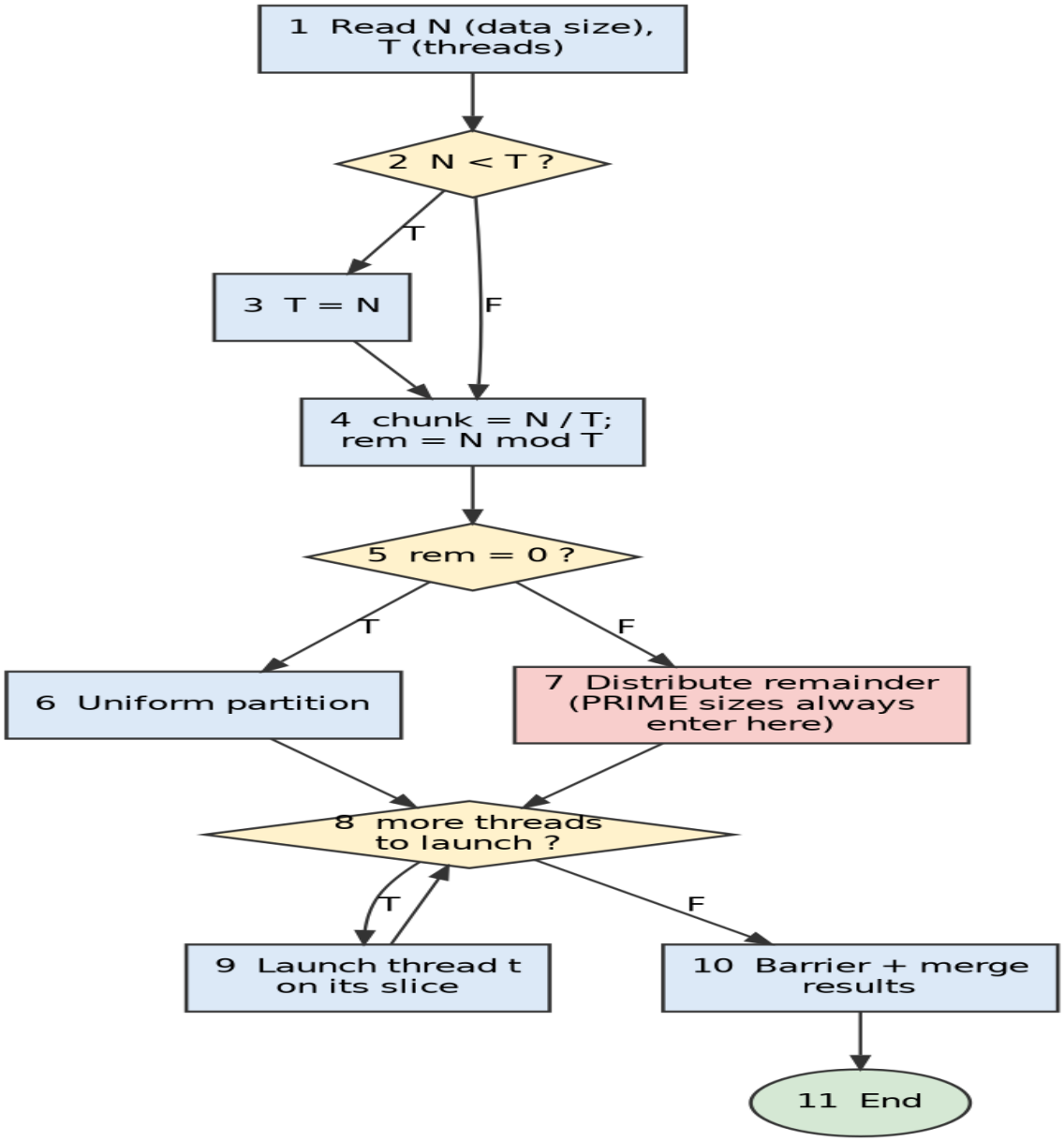


Fig 15.1 CFG of the parallel partitioning routine

2. Structured White-Box Approach with Basis Path Testing

V(G) = E - N + 2 = 13 - 11 + 2 = **4** (decisions 2, 5, 8 -> 3 + 1). Four basis paths:

Path	Node sequence	Scenario
P1	1-2-4-5-6-8-9-8-10-11	N >= T, N divisible by T (even split)
P2	1-2-3-4-5-6-8-9-8-10-11	N < T: thread count reduced to N

Path	Node sequence	Scenario
P3	1-2-4-5-7-8-9-8-10-11	N >= T, remainder exists (**prime sizes**)
P4	1-2-4-5-6-8-10-11	Loop never entered (T = 0): defensive path, input must be rejected

3. Justification: Mathematical Accuracy and System Stability

- **Targets the real cause:** basis paths force the remainder branch (P3) to run, which random "round" sizes never did; the prime-size defect is reproduced deterministically by M3-M5.
- **Mathematical accuracy:** the differential oracle plus invariants prove every element is processed exactly once and results match the reference within a defined tolerance.
- **Stability:** loop-boundary, defensive-path (T = 0) and repeated concurrent runs expose crashes, hangs and races under all slicing shapes.
- **Measurable adequacy:** 100% of the 4 independent paths, 100% branch coverage and all loop boundaries, with only about 20 tests instead of an unbounded number of random runs. In medical imaging, a verified and reproducible result is also a regulatory expectation (e.g. IEC 62304).

Conclusion: replacing random testing with CFG-driven basis path testing, invariant checks and a serial-reference oracle makes the module's correctness independent of whether the data size happens to be prime, and gives auditable evidence of accuracy and stability.

Code of Conduct:

*I **BHUVANESHWARI B A (192421195)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
BHUVANESHWARI B A

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification